系统优化与鲁棒性——讲义

方 洲

2026 年 4 月 16 日

目录

2 系统最优控制的数值方法 **2**

2.1 线性系统的数值方法 2

2.2 非线性系统的数值方法–强化学习 6

2　系统最优控制的数值方法

2.1　线性系统的数值方法

这节课我们尝试回答这样一个问题：在机器算力日益强大的今天，我们似乎可以通过直接数值计算来“暴力求解“最优控制问题。那么，为什么仍然需要学习较为复杂的最优控制理论？它对于实际系统的分析与调控究竟有什么作用？

思考一下，答案在小节2.1.3。

为了回答这个问题，我们需要在不同案例中比较直接计算方法与基于最优控制理论的数值方法之间的差别。在比较中，我们考虑线性系统：

$$\dot{x}(t) = Ax(t) + Bu(t), \qquad x(0) = x_0,$$

其中，$x(t) \in \mathbb{R}^n$、$u(t) \in \mathbb{R}^m$、A 与 B 分别为矩阵。给定初始状态 x_0 的条件下，我们寻找控制函数 $u(t)$，最小化性能指标

$$J(u) = \int_0^T \left(x(t)^\top Q x(t) + u(t)^\top R u(t)\right) \mathrm{d}t + x(T)^\top F x(T)$$

其中，$Q \succeq 0$、$R \succ 0$、$F \succeq 0$ 为权重矩阵。

2.1.1　基于梯度下降的直接计算方法

一种最直接的思路是：先对上述连续问题进行离散化，再把每个离散时刻的控制输入都看作待优化变量，从而直接对离散化后的目标函数进行数值优化。为避免记号过于繁杂，下文仍用 x 和 u 分别表示离散时间系统的状态序列与控制序列。

具体而言，当时间区间 $[0, T]$ 被划分为足够多的 N 个步长（即时间离散足够稠密），我们可以用如下的离散时间系统近似原系统

$$x(t_{k+1}) = x(t_k) + [Ax(t_k) + Bu(t_k)]\,\Delta t, \qquad k = 0, 1, \ldots, N-1$$

此时，优化变量为整个离散的控制序列 $u = \big(u(t_0), u(t_1), \ldots, u(t_{N-1})\big)$，待优化的目标

函数是

$$J(u) = \sum_{k=0}^{N-1} \left[x(t_k)^\top Q x(t_k) + u(t_k)^\top R u(t_k) \right] \Delta t + x(t_N)^\top F x(t_N)$$

很容易验证这依然是一个凸优化问题，从而我们可以利用标准的梯度下降法求得该问题的最优解。对应的梯度下降迭代公式为

$$u^{(m+1)} = u^{(m)} - \eta_m \nabla J\big(u^{(m)}\big)$$

其中，$u^{(m+1)}$ 为第 m 步的迭代结果，η_m 为第 m 步的步长。

其主要差距在于计算时间方面

2.1.2 基于 Riccati 方程的计算方法

之前的课程内容提到二次最优控制器满足

$$u^*(t) = -R^{-1} B^\top P(t)\, x(t) \tag{2.1}$$

其中，矩阵 $P(t)$ 是下述 Riccati 方程的解。

$$-\dot{P}(t) = A^\top P(t) + P(t) A - P(t) B R^{-1} B^\top P(t) + Q, \qquad P(T) = F.$$

Riccati 方程本质上是一个终端约束的常微分方程。从而，我们只需对 Ricatti 方程进行数值求解，并代入 u^* 的表达式(2.1)，即可求解该问题。

2.1.3 两种方法的数值比较

“最优控制理论导出了更高效的最优控制器求解方法。”

我们通过数值实验比较了基于梯度下降的直接方法与基于 Riccati 方程的方法；相关实现代码见脚注[1]。在数值实验中，我们设定：

$$A = \begin{bmatrix} -0.7 & 0.5 \\ 1 & -1.5 \end{bmatrix}, \quad B = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R = \begin{bmatrix} 1 \end{bmatrix}, \quad F = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},$$
$$x_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \qquad\qquad T = 10$$

[1] https://github.com/ZhouFang92/Systems-Optimization-and-Robustness/blob/main/lqr_method_comparison.ipynb

两种方法在最优控制器设计和性能指标方面表现相近（见 Figure 2.1）。主要差别体现在计算时间上，详见表 2.1。与基于梯度下降的直接方法相比，基于 Riccati 方程的方法的计算效率提升了 50 倍以上。

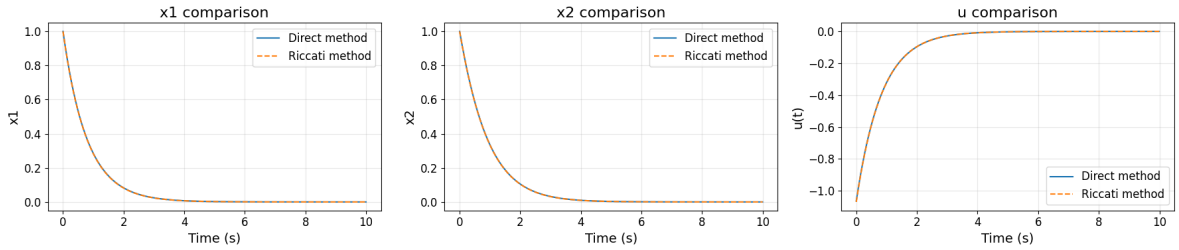

Figure 2.1: 直接方法与基于最优控制理论的方法在数值计算中的比较示意图

Table 2.1: Apple M1 Pro 芯片环境下两种方法的计算时间比较

方法	计算时间（秒）	相对速度
基于梯度下降的直接方法	约 3.00	$1\times$
基于 Riccati 方程的方法	约 0.05	$> 50\times$

2.1.4 原因的进一步说明

我们进一步探究：为什么基于 Riccati 方程的方法具有更高的计算效率？一种自然的解释是，理论方法给出了更好的解的结构，因此能够提升计算效率。这一说法在一定程度上是成立的。然而，在一些问题中，基于显式表达式的计算并不一定会带来更高的效率，有时甚至可能显著增加计算代价。因此，我们还需要更明确地理解：Riccati 方程究竟是如何提升计算效率的？

我们给出一个较为直观的解释。这里考虑的最优控制问题本质上仍然是一个最优化问题，可以写成

$$\min_{u} \; u^\top \tilde{R} u + \tilde{F}^\top u$$

其中，$u \in \mathbb{R}^{L_u}$ 表示控制序列构成的向量；$\tilde{R} \in \mathbb{R}^{L_u \times L_u}$ 表示在综合考虑系统动力学约束与代价泛函之后得到的等效二次型权重矩阵；$\tilde{F} \in \mathbb{R}^{L_u}$ 表示相应的等效一次项系数向量。$\tilde{R}$ 与 $\tilde{F}$ 的具体表达式均可自行推导。

- 直接方法的计算复杂度：基于梯度下降的直接方法，其总计算复杂度可以理解为“优化迭代步数”与“单次梯度计算复杂度”的乘积，因此可近似写为 $\mathcal{O}\big(\text{优化迭代步数} \times L_u^2\big)$，其中 L_u 表示控制序列 u 的长度。

- 线性方程求解法：如果进一步利用二次型结构，则最优性条件可以转化为一个规模为 L_u 的线性方程组求解，即 $\tilde{R}u^* = F$。其计算复杂度通常约为 $\mathcal{O}(L_u^3)$。从

更一般的角度看，最大值原理本质上也是为最优控制问题给出相应的最优性条件，即一些微分方程形式的条件。

在上述分析中，两种方法的优劣核心在于控制序列长度 L_u 与迭代步数的比较，在不同情况下各有优劣。当控制序列特别长的时候，直接方法甚至于超越第二中方法。所以，Ricatti 方法的好处并非来源于一般的最优性条件。

上述分析并未考虑最优控制问题的时序结构。增加时序性质以后，两种方法的算法效率都有提升。而第二种方法涉及到矩阵求逆，复杂度与矩阵规模呈现 3 次方效果，故而放大效果更为显著。

- 时序情况下的直接方法：其总计算复杂度依然是“优化迭代步数”与“单次梯度计算复杂度”的乘积。增加时序性质以后，单次 J 函数的计算复杂度为 $\mathcal{O}(n^2 \times N)$，其中 N 为时间序列的长度。类似的，利用反向传播（backward propagation）单次求梯度的时间复杂度也是相同的。从而，总体复杂度是 $\mathcal{O}(\text{优化迭代步数} \times n^2 \times N)$。

- 基于 Ricatti 方程的方法：利用时序信息，$\tilde{R}$ 的求逆的过程可以分解为 Ricatti 方程求解，与最优控制计算两步。Ricatti 方程求解的复杂度为 $\mathcal{O}\left(n^3 \times N\right)$；第二步的复杂度也是 $\mathcal{O}\left(n^3 \times N\right)$。所以，总体复杂度为 $\mathcal{O}\left(n^3 \times N\right)$。

可以看到，此时真正决定两种方法优劣的是状态维数 n 与优化迭代步数之间的比较。通常优化迭代步数不少于 100 次，很多情形下甚至可达上万次。因此，只要系统的维数不是特别高，基于 Riccati 方程的方法具有明显的计算优势。

在具体应用中，最优控制器的计算往往是许多控制模块的基础组成部分，并且会被大量重复调用。因此，计算效率在实际问题中具有重要意义。

在上述分析中，我们只考虑了初始点固定的情形。当初始点 x_0 不确定时，直接方法往往需要针对不同初始状态重复进行优化，单次梯度下降的计算复杂度将更加巨大。而理论方法则更容易复用已有结果，因此其优势通常会更加明显。

2.2 非线性系统的数值方法–强化学习

我们尝试回答另外一个问题：这门课程的许多最优控制理论主要针对线性系统建立，那么面对非线性系统时，我们应该如何进行数值计算?

我们依然可以利用动态规划、最大值原理等理论构建数值方法，但相应的计算复杂度通常非常高。这是因为：最优控制器往往不再具有十分简洁的显式表达式，我们需要在每一个时刻 t 和每一个状态点 x 上确定相应的控制决策，因此存储和计算开销都十分巨大。解决这类计算难题通常需要引入一定的近似，其总体思路大致可以分为两类：

1. 先对问题进行简化（例如线性化），再进行数值求解。例如，在 MPC 中，可以先将系统作局部线性化，再基于线性系统的最优控制问题进行求解。
2. 利用动态规划等一般性理论构建算法，再借助神经网络等近似技术进行求解。若问题规模较小，也可以直接计算。

第一类思路在前面的内容中已经有所涉及，且较容易自行推导，因此这里不再赘述。本节课主要介绍第二类思路中的一种方法——利用强化学习中的策略学习（policy learning）来求解最优控制问题中的控制策略。

2.2.1 策略学习的框架

考虑如下非线性系统以及目标函数：

$$
\begin{aligned}
&\dot{x}(t) = f\big(x(t), u(t)\big), \qquad x(0) = x_0 \in \mathcal{X}_0 \subseteq \mathbb{R}^n \\
&J(x_0, u) = \int_0^T \ell\big(x(t), u(t)\big)\,\mathrm{d}t + \Phi\big(x(T)\big).
\end{aligned}
$$

我们假设对于不同的初始状态 x_0，都存在由同一反馈律表示的最优控制策略 $u^*(t,x)$；我们的任务就是找到这一最优策略，或它的一个近似结果。

策略学习的核心思想是利用神经网络 $\pi_\theta(t,x)$（θ 为神经网络的权重参数）表示控制策略 $u(t,x)$，从而将最优控制问题参数化：

$$
\begin{aligned}
&\dot{x}(t) = f\big(x(t), \pi_\theta(t,x)\big), \qquad x(0) = x_0 \in \mathcal{X}_0 \subseteq \mathbb{R}^n \\
&\tilde{J}(x_0, \theta) \triangleq \int_0^T \ell\big(x(t), \pi_\theta(t, x(t))\big)\,\mathrm{d}t + \Phi\big(x(T)\big)
\end{aligned}
$$

并通过求解下述优化问题来确定最优参数 θ，从而近似最优控制 u^*：

$$
\min_\theta \quad \underbrace{\frac{1}{\mathcal{X}_0\text{的测度}} \int_{x_0 \in \mathcal{X}_0} \tilde{J}(x_0, \theta)\mathrm{d}x_0}_{J^{\mathrm{tot}}(\theta)} \tag{2.2}
$$

2.2.2 目标函数可导情形下的策略学习框架

当问题中涉及的函数 f、ℓ、Φ 都可导时，性能函数 $\tilde{J}(x_0,\theta)$ 关于 θ 也是可导的。那么，我们可以直接使用梯度下降法等优化方法来学习最优的 $\pi_\theta(t,x)$。这与前面线性系统部分介绍的直接方法在思想上是一致的。

需注意，我们无法对所有的初始点进行遍历，需使用蒙特卡洛方法计算优化目标(2.2)。即将该优化目标改写为

$$J^{\mathrm{tot}}(\theta)=\mathbb{E}_{x_0\sim U(\mathcal{X}_0)}\left[\tilde{J}(x_0,\theta)\right] \qquad U(\mathcal{X}_0)\text{ 为集合 }\mathcal{X}_0\text{ 上的均匀分布}$$

对应的梯度为

$$\nabla J^{\mathrm{tot}}(\theta)=\mathbb{E}_{x_0\sim U(\mathcal{X}_0)}\left[\nabla\tilde{J}(x_0,\theta)\right]$$

在实际的每次优化迭代中，具体计算为

1. 从集合 $\mathcal{X}_0$ 上随机采样一批初始状态 $x_0^{(1)},\dots,x_0^{(M)}$；
2. 对每个样本 $x_0^{(i)}$，在当前参数 $\theta^{(m)}$ 下正向求解闭环系统，得到相应的状态轨迹 $x^{(i)}(\cdot)$；
3. 计算经验目标函数的梯度

$$\nabla J_M\big(\theta^{(m)}\big)=\frac{1}{M}\sum_{i=1}^{M}\nabla\tilde{J}\big(x_0^{(i)},\theta^{(m)}\big);$$

4. 按照更新公式

$$\theta^{(m+1)}=\theta^{(m)}-\eta_m\nabla J_M\big(\theta^{(m)}\big)$$

数值的案例见[2]。

2.2.3 目标函数不可导情形下的策略学习框架

当系统动力学、目标函数或仿真环境中含有不可导环节时，我们往往无法像上一小节那样直接对 $\tilde{J}(x_0,\theta)$ 关于 θ 进行求导。此时，我们需要稍微改造问题，从而实现对目标函数的求导。

一种常见做法是将确定性策略推广为随机策略 $\pi_\theta(t,x)$。例如，在连续控制中，常

[2] https://github.com/ZhouFang92/Systems-Optimization-and-Robustness/blob/main/RL_for_derivative_objective.ipynb

将策略建模为高斯分布

$$\pi_\theta(x,t) \sim \mathcal{N}\big(\mu_\theta(x,t), \Sigma_\theta(x,t)\big).$$

其中，$\mu_\theta(x,t)$ 表示策略的均值函数，$\Sigma_\theta(x,t)$ 表示策略的协方差矩阵；在常见设定下，它通常取为对角矩阵。在离散化后的时间网格上，记整段状态序列与控制序列为

$$x = (x_0, x_1, \ldots, x_N) \qquad\qquad u = (u_0, u_1, \ldots, u_{N-1})$$

在给定初始状态 x_0 的条件下，整段控制序列的条件概率密度可写为

$$\begin{aligned} p_\theta(u \mid x_0) = \prod_{k=0}^{N-1} & \frac{1}{\sqrt{(2\pi)^m \det \Sigma_\theta(x_k,t_k)}} \\ & \times \exp\left(-\frac{1}{2}\big(u_k - \mu_\theta(x_k,t_k)\big)^\top \Sigma_\theta(x_k,t_k)^{-1}\big(u_k - \mu_\theta(x_k,t_k)\big)\right). \end{aligned}$$

x_0 与 u 的联合分布可写为

$$p_\theta(x_0,u) = p_{X_0}(x_0)\, p_\theta(u \mid x_0), \qquad p_{X_0}(x_0) = \frac{1}{\mathcal{X}_0\text{的测度}}, \quad x_0 \in \mathcal{X}_0.$$

于是，总体目标函数可写为

$$J^{\mathrm{tot}}(\theta) = \mathbb{E}_{(x_0,u)\sim p_\theta(x_0,u)}\big[J(x_0,u)\big].$$

此时，我们实际上改变了原问题的寻优空间：原先是在确定性控制策略类中寻找最优解，现在则是在随机策略类中进行优化。这样做的好处在于，即便 $J(x_0,u)$ 本身不可导，只要联合分布 $p_\theta(x_0,u)$ 关于参数 θ 可导，我们仍然可以对总体目标函数 $J^{\mathrm{tot}}(\theta)$ 求导。具体地，

$$\begin{aligned} \nabla_\theta J^{\mathrm{tot}}(\theta) &= \nabla_\theta \int J(x_0,u)\, p_\theta(x_0,u)\, \mathrm{d}x_0\, \mathrm{d}u \\ &= \int J(x_0,u)\, \nabla_\theta p_\theta(x_0,u)\, \mathrm{d}x_0\, \mathrm{d}u \\ &= \int J(x_0,u)\, p_\theta(x_0,u)\, \nabla_\theta \log p_\theta(x_0,u)\, \mathrm{d}x_0\, \mathrm{d}u \\ &= \mathbb{E}_{(x_0,u)\sim p_\theta(x_0,u)}\left[J(x_0,u)\nabla_\theta \log p_\theta(x_0,u)\right]. \end{aligned}$$

在实际计算中，上述期望通常通过蒙特卡洛方法来近似。由于上述随机变量 $J(x_0,u)\nabla_\theta \log p_\theta(x_0,u)$ 的方差可能非常大。为了减少误差，我们常常会在代价前减去

一个与控制输入无关的基线项 b，即

$$\nabla_\theta J^{\mathrm{tot}}(\theta) = \mathbb{E}_{(x_0,u)\sim p_\theta(x_0,u)}\left[(J(x_0,u)-b)\,\nabla_\theta \log p_\theta(x_0,u)\right].$$

这样不会改变梯度估计的期望，但通常可以显著降低梯度估计的方差，从而提高算法的稳定性。这种处理在统计上可视为一种控制变量（control variate）技术；在强化学习文献中，通常也称为引入基线（baseline）。此时，b 通常选取为随机变量 $J(x_0,u)$ 的均值或其近似值。

设第 m 次迭代时的参数为 $\theta^{(m)}$。在使用基线项 b 的情况下，可按如下步骤进行：

1. 从集合 $\mathcal{X}_0$ 上随机采样 M 个初始状态 $x_0^{(1)},\dots,x_0^{(M)}$；
2. 对每个初始状态 $x_0^{(i)}$，按照当前随机策略 $\pi_{\theta^{(m)}}$ 逐步采样控制输入，得到控制序列 $u^{(i)}$ 与状态序列 $x^{(i)}$；并计算每条样本轨迹对应的代价

$$J^{(i)} = J\big(x_0^{(i)},u^{(i)}\big), \qquad i=1,2,\dots,M;$$

3. 构造梯度估计

$$\hat{g}^{(m)} = \frac{1}{M}\sum_{i=1}^{M}\big(J^{(i)}-b\big)\nabla_\theta \log p_{\theta^{(m)}}\Big(x_0,u_k^{(i)}\Big);$$

4. 按照更新公式

$$\theta^{(m+1)} = \theta^{(m)} - \eta_m \hat{g}^{(m)}$$

数值的案例见[3]。

这类基于策略优化的强化学习方法，最早是在马尔可夫决策过程与随机最优控制问题中逐步发展起来的。后来，这种“将复杂问题表述为多步动态决策问题，并直接优化策略”的思想被广泛推广到博弈（如 AlphaGo）、自然语言处理、符号回归、自动定理证明等任务中，构建起了当今人工智能的基石。大家所学的系统控制理论并不只服务于传统控制问题，它也完全可能在其它领域发挥关键作用，甚至带来革命性的突破。

课程作业：

以自己感兴趣的系统为例，运用所学的最优控制理论进行分析，并展示数值结果。提交的报告应为 PDF 格式，可附补充材料；作业提交情况将计入课程成绩。

[3] https://github.com/ZhouFang92/Systems-Optimization-and-Robustness/blob/main/RL_for_non_derivative_objectives.ipynb